ChatGPT

Pickaxe Capital Recovery Blueprint and Codex Prompts

Executive summary

Because I could not directly inspect your repository in this conversation, the most reliable deliverable is not a pretend code audit; it is a recovery-first architecture and prompt package that forces Codex to inspect the live files, identify the actual framework and breakpoints, then stabilize the project before doing anything cosmetic or ambitious. That approach matches OpenAI's current Codex guidance: Codex performs best when the task has explicit success criteria, is broken into smaller focused steps, and follows a disciplined loop of planning, editing, running checks, observing failures, repairing them, and repeating until the repo is genuinely verified. OpenAI also states that Codex produces higher-quality output when it can validate its work with real commands such as build, lint, and tests. [1]

Your final product vision should be simpler than the prompt history that created the current mess. The site should ship as a **premium command-center website**, not as a fake autonomous operating system. The clean hierarchy is: **Pickaxe Capital** as the public company brand, **AI Habitat OS** as the internal operating system, and **CEO B** as the command layer. Everything else should become either a route, a subsystem, or a clearly labeled research/prototype concept. The homepage should explain that hierarchy in seconds, while the rest of the routes deepen it rather than multiplying it.

The external repos you named are useful precisely because they show why your site should **not** pretend to be one integrated live system yet. OpenClaw is described as a **self-hosted gateway** and single routing/control layer across chat surfaces, not a marketing website. Addy Osmani's Jarvis is a **React multimodal live assistant** with real audio/video and tool services. `jarvis-mlx` explicitly warns that it is **very out of date**. OpenJarvis positions itself as **local-first personal AI** that should run on personal devices and call the cloud only when necessary. Microsoft's JARVIS is a research workflow organized around **task planning, model selection, execution, and response generation**. CraftJarvis JARVIS-1 is an **open-world multimodal Minecraft agent** with memory. Those are different runtime targets, different maturity levels, and different product surfaces, so the safe web strategy is to translate them into **research cards, concept diagrams, future adapters, and clearly labeled prototypes** rather than fake live claims. [2]

The working rule for Codex should therefore be simple: **recover first, consolidate second, polish third, and only then consider new capabilities**. Anything that is not implemented and testable should be labeled **Static**, **Prototype**, **Research**, or **Future Adapter**. That single honesty layer will immediately make the site feel more premium, because premium products do not bluff.

A strict do-not-do list for Codex should be part of the recovery contract:

- Do **not** migrate frameworks during recovery unless the existing build is irreparably miswired.
- Do **not** copy external GitHub repos or transplant their code.

- Do **not** invent live voice, live camera, live device control, autonomous agents, trading execution, or any other theatrical demo state.
- Do **not** preserve duplicate route trees, duplicate shells, duplicate components, or duplicate data sources.
- Do **not** add more features until the existing build is green.
- Do **not** stop after writing a plan; complete real edits, run real checks, and report exact outcomes. [3]

Diagnosis framework

The highest-probability failure pattern is not “bad taste”; it is **architecture drift caused by repeated ambitious prompts without a strong single source of truth**. React’s docs warn that redundant or duplicate state is a common source of bugs, that state should be owned in one place and lifted upward when multiple components need it, that components should stay pure, and that Effects are only for synchronizing with external systems rather than deriving normal UI state. Separately, both Next.js and React Router provide shared layout patterns specifically so pages do not need to keep re-creating shells and route scaffolding. When an AI-built project ignores those guardrails, you usually get duplicated pages, contradictory content, broken imports, unnecessary effects, and brittle route behavior. [4]

A useful forensic framework for your repo is this:

Likely failure mode	What it usually looks like	Recovery move
Brand drift	Multiple names used as if each is a separate product, not part of one system	Create one canonical naming map and apply it everywhere
Route sprawl	Several pages do the same job, optional labs shadow core pages, dead routes remain linked	Keep core routes, simplify or quarantine optional routes
Shell duplication	Home page, lab pages, and profile pages each have their own nav/header/card system	Replace with one shared site shell and one page-header pattern
Component clone drift	Multiple versions of hero cards, status cards, command consoles, grids, badges	Consolidate to shared primitives and delete shadow copies
Data duplication	Nav labels, route metadata, status strings, and content blocks hard-coded in many files	Move route/meta/content into central typed data files
Effect abuse	Local filtering, derived display state, or simple toggles handled through `useEffect` chains	Compute during render or via event handlers; reserve Effects for actual external sync
Prototype inflation	UI implies voice, agents, devices, trading, or live systems that are not implemented	Add honesty labels and reduce to static/prototype adapters

Likely failure mode	What it usually looks like	Recovery move
Style drift	Mixed gradients, unreadable glassmorphism, random typography, inconsistent spacing	Centralize design tokens and apply one premium system
Verification gap	“Looks okay” in generated code, but build/lint/typecheck were never run cleanly	Make build verification a non-negotiable stopping rule

The right order of operations is also important. OpenAI’s Codex docs explicitly favor a loop where the agent plans, edits, runs tools, observes failures, repairs them, and only then stops. So the repo should be recovered in this order: **detect stack and entrypoints, fix build errors, stabilize router and shell, consolidate duplicates, simplify broken pages, then polish**. [5]

Final site architecture

The strongest achievable version of this project is a **single command-center shell** with a persistent left navigation, a clear top header/state bar, a main content panel, and an optional right-side context rail on desktop. That layout feels “operational” without pretending to be a live control room, and it maps neatly to accessible document landmarks such as `navigation`, `main`, and `aside` / `complementary`. If the app is already on Next.js, a shared layout fits naturally into `layout.tsx`. If it is on React Router, a single layout route should wrap the route tree without changing URLs. [6]

The route architecture should be this:

Route	What it is	What should ship now
`/`	Public command-center landing page	Brand hierarchy, system overview, quick links to core routes, concise current-status panels
`/vision-map`	Map of the whole system	Static architecture diagram showing Pickaxe Capital → AI Habitat OS → CEO B → Agents / Archive / Staging / Labs / Devices
`/agents`	Visual AI worker habitat	Grid/list of agent cards with role, inputs, outputs, tool scope, status, and next action
`/archive`	Intelligence vault	Searchable/filterable local dataset of links, repos, research notes, sources, and next actions
`/staging`	Progress and QA center	Build status, route status, known issues, cleanup log, and next steps
`/founder`	Founder narrative	Vision, thesis, principles, and why Pickaxe Capital exists
`/ceo-b-profile`	CEO B profile	Command-layer profile page explaining decision model, device roles, operating principles

Route	What it is	What should ship now
`/jarvis-lab`	Optional lab	Typed command console only, with safe local commands like navigation, panel toggles, and archive search
`/life-os`	Optional future overview	Device strategy page for phone, iPad, Mac, and desktop roles
`/device-hub`	Optional future adapter hub	Device cards, integration notes, local/offline roadmap, no fake device control
`/vision-lab`	Optional experiments gallery	Research visuals, mockups, system concepts, and future experiments

The key architectural rule is that **core routes must always resolve**, even if some have to be temporarily simplified. A broken ambitious page is worse than a clean, honest page. If an optional route already exists and is stable, keep it. If it is present but broken, convert it into a minimal in-shell page with a clear **Prototype**, **Research**, or **Future Adapter** label instead of deleting it or keeping it theatrically half-working.

The labeling system should be standardized across the entire site:

- **Live**: implemented and testable now.
- **Static**: informational, no runtime integration.
- **Prototype**: local UI behavior only; not wired to external systems.
- **Research**: inspiration/reference only.
- **Future Adapter**: planned integration contract or device bridge; not active yet.

That labeling scheme is especially important because the repos you cited should be translated, not re-enacted. OpenClaw belongs as a **future integration gateway** concept, not as a fake web-native assistant backend. Addy Osmani's Jarvis belongs as **UI inspiration** for `/jarvis-lab`, but your site should not imply real voice/camera control unless those services truly exist. `jarvis-mlx` is explicitly outdated, which supports treating the Mac offline brain idea as **future research**. OpenJarvis and Microsoft JARVIS are strongest as **conceptual architecture references** on `/vision-map`: local-first operation, with controller/planner layers and careful routing of tasks. JARVIS-1 fits best as inspiration for the **visual grammar of agents and memory**, not as a claim that your site ships embodied-world control. [2]

File structure recommendation

Your repo should move toward a structure where **content/data lives in one place, layout lives in one place, and presentation is built from reusable primitives**. React's guidance is straightforward here: duplicate or contradictory state is a bug magnet, shared state should have one owner, components should stay pure, and Effects should only be used when synchronizing with real external systems. That makes a strong case for central route metadata, central design tokens, local typed content modules, and dumb presentational components wherever possible. [7]

Use this as the target shape, mapped onto whatever framework the repo already uses:

```
app/ or src/
  routes/ or app/
    home
    vision-map
    agents
    archive
    staging
    founder
    ceo-b-profile
    jarvis-lab
    life-os
    device-hub
    vision-lab

components/
  layout/
    SiteShell
    SideNav
    TopBar
    PageHeader
    ContextRail
  primitives/
    Card
    Panel
    Badge
    Section
    Stat
    EmptyState
    SearchInput
  blocks/
    HeroPanel
    StatusGrid
    SystemMap
    AgentCard
    ArchiveTable
    CommandConsole
    DeviceMatrix

content/
  brand.ts
  routeMeta.ts
  navigation.ts
  status.ts
  agents.ts
  archive.ts
  staging.ts
  devices.ts
```

```
  founder.ts

lib/
  cn.ts
  format.ts
  routeHelpers.ts
  constants.ts

styles/
  globals.css
  tokens.css
  utilities.css

tests/
  smoke/
  components/
```

The most important central files are not glamorous. They are `navigation`, `routeMeta`, `status`, and the content registries for agents, archive items, and staging items. If your nav labels, badge semantics, or route descriptions are duplicated inside each page, Codex should move them out. Once that happens, the site becomes much easier to finish without drift.

For styling, use a single token layer. Tailwind's current docs explicitly frame theme variables as the project's design tokens and allow colors, fonts, spacing, radius, shadows, and breakpoints to be centralized in one place. If the project is already Tailwind-based, keep one token source and stop the scattering of arbitrary values. If it is plain CSS, use one `tokens.css` file with custom properties. Either way, the goal is the same: one spacing scale, one type scale, one panel style, one accent strategy, one motion strategy. [8]

If the project already uses TypeScript, keep or tighten strictness rather than loosening it for convenience. TypeScript's `strict` mode exists specifically to provide stronger guarantees of correctness, and `strictNullChecks` prevents the common "this was assumed to exist" runtime failures that plague generated code. If ESLint is already present, keep its config active; ESLint's purpose is to surface patterns that reduce consistency and increase bug risk. [9]

Recovery Codex prompt ready to copy

This prompt is designed to match what OpenAI says Codex handles best: clear success criteria, smaller focused steps, explicit verification, and a real "done when" standard instead of a vague plan. It tells Codex to inspect first, fix build errors first, simplify before expanding, and end with exact verified results. [10]

```
You are working inside my existing website repository.

This is a recovery-first refactor of an existing project, not a greenfield
build.
Do not invent a new app. Do not redesign the product scope. Do not add
```

```
speculative features.

PRIMARY GOAL
Stabilize, simplify, and finish the current Pickaxe Capital / AI Habitat OS
website so it becomes a clean, premium, readable command-center website that
builds successfully and preserves the best working parts.

NON-NEGOTIABLE RULES
- Inspect the current files first before making major changes.
- Detect the real stack from the repo itself: package.json, lockfile, tsconfig/
jsconfig, framework config, router setup, app/src structure, styles, and
scripts.
- Use the framework/router already wired into the repo. Do NOT migrate Next.js
to Vite or Vite to Next.js unless the current app is completely nonfunctional
and unrecoverable.
- Fix build errors first.
- Preserve working routes.
- If a route is broken, simplify it into a clean stable version instead of
deleting it.
- Remove duplicates, dead files, conflicting route trees, unused experimental
copies, and shadow components.
- Do NOT copy code from external GitHub repos.
- Do NOT fake live AI, live voice, live camera, live device control, autonomous
agents, or trading execution.
- Any advanced capability that is not truly implemented must be labeled clearly
as Prototype, Research, or Future Adapter.
- Run the build before stopping.
- If lint and/or typecheck scripts exist, run them after the build is green.
- End with an exact report of file changes, deletions, route status, and build
results.

PROJECT IDENTITY
Use this naming hierarchy consistently:
- Pickaxe Capital = company / public brand
- AI Habitat OS = internal operating system
- CEO B = command layer / decision brain

TARGET ROUTES

Core routes that must exist cleanly:
- /
- /vision-map
- /agents
- /archive
- /staging
- /founder
- /ceo-b-profile
```

```
Optional routes only if already present or safe:
- /jarvis-lab
- /life-os
- /device-hub
- /vision-lab

If an optional route exists but is unstable, keep the route and reduce it to a
safe simple page inside the shared shell with honest status labels.

FINAL PRODUCT VISION
This website is a premium command-center website, not a fake autonomous
operating system.
It should feel clean, deliberate, structured, and impressive.
It should prioritize clarity, hierarchy, readability, and honest status labeling
over hype.

STATUS LABELS
Use these labels consistently where relevant:
- Live = implemented and testable now
- Static = informational only
- Prototype = local UI demo only, no real external execution
- Research = inspiration / concept / reference
- Future Adapter = planned integration, not active

ROUTE INTENT

/
- Public-facing command-center home
- Explain the hierarchy: Pickaxe Capital -> AI Habitat OS -> CEO B
- Include quick route cards linking to all core routes
- Include concise status panels
- Eliminate clutter and repeated brand names

/vision-map
- Show the whole system map
- Visualize relationships among company, OS, CEO B, agents, archive, staging,
labs, and devices
- Static/SVG/CSS visualization is enough
- No fake live graph or fake telemetry

/agents
- Show visual AI worker cards / agent habitat
- Each card should have: name, role, inputs, outputs, tools/scope, status, and
next action
- No fake autonomous execution
- If current agent content is messy, simplify into a clean roster

/archive
```

```
- Make this the intelligence vault
- Use local data or existing content in the repo
- Store/render links, bookmarks, repos, research sources, notes, and next
actions
- Search/filter UI is okay if safe and local
- Do not require a backend unless one already exists and works

/staging
- Make this the progress tracker and QA page
- Show current route status, known issues, cleanup progress, and next steps
- This can be driven by local static data/constants for now
- No fake CI integrations

/founder
- Clean founder page with mission, thesis, and principles
- Keep it readable and premium
- Avoid bloated manifesto styling

/ceo-b-profile
- Make this the command-layer profile
- Explain CEO B as the decision brain / command layer
- Show device-role summary and operating principles
- Do not personify it as a live omniscient AI if that is not implemented

Optional routes:

/jarvis-lab
- Typed command console only
- Safe behaviors: route navigation, archive search, panel toggles, local command
history, static command suggestions
- Explicitly label voice/camera as future capabilities if not implemented
- No fake microphone/camera control

/life-os
- Device role overview for phone, iPad, Mac, desktop
- Clear static strategy page is enough

/device-hub
- Device matrix / integration adapters / future device strategy
- No fake control center behavior

/vision-lab
- Experimental visual concepts and research mockups
- Keep stable and minimal

MANDATORY IMPLEMENTATION ORDER

1. AUDIT FIRST
```

```
- Inspect package.json scripts, config files, entry points, route files,
component folders, styles, and content/data folders
- Determine the real framework, router, build command, and active source root
- Identify:
  - build/type/import/runtime blockers
  - duplicate pages or components
  - dead experimental files
  - repeated nav/data/status content
  - broken routes
  - inconsistent naming

2. FIX THE BUILD FIRST
- Run the existing build command from package.json
- Capture and fix compile/import/type/build errors before doing cosmetic work
- If there are multiple app trees, keep the one actually used by the build and
remove or isolate dead duplicates

3. CONSOLIDATE STRUCTURE
- Create or preserve one shared site shell
- Create one canonical navigation source
- Create one canonical route metadata source
- Consolidate repeated cards/panels/badges into shared reusable primitives
- Remove shadow copies and unused variants
- Keep code simple and easy to reason about

4. RECOVER ROUTES
- Ensure all core routes render cleanly
- Preserve working content where helpful
- Simplify broken pages instead of making them more ambitious
- Optional routes should remain only if safe

5. CONTENT HONESTY PASS
- Replace fake live wording with honest statuses
- Label advanced concepts as Prototype / Research / Future Adapter where needed
- Do not claim live agents, live device control, or live trading if not real

6. VISUAL QUALITY PASS
- Make the UI premium, readable, and consistent
- Improve hierarchy, whitespace, typography, card consistency, spacing, and
navigation
- Keep styling restrained and high-signal
- Avoid random gradients, unreadable glassmorphism, cramped layouts, and over-
animation
- Respect mobile and desktop layouts

7. VALIDATION
- Run build again
- Run lint if available
```

```
- Run typecheck if available
- If a lightweight existing test setup already exists, run the relevant tests
- Do not install a large new testing framework during emergency recovery unless
absolutely necessary to fix the repo

COMPONENT / DATA / CSS RULES
- Prefer one source of truth for navigation, route metadata, statuses, and
content collections
- Prefer local typed data/content files over duplicated strings across pages
- Keep components pure and simple
- Avoid unnecessary effects for derived UI state
- Use one token system for colors, spacing, radius, shadows, and typography
- Keep badges/status chips consistent across all routes

DO NOT DO
- Do not create a brand-new parallel app
- Do not introduce a backend just to simulate progress
- Do not add MCP/tool integrations
- Do not add websockets, camera APIs, mic APIs, or device APIs unless already
implemented and working
- Do not add stock/trading dashboards that imply real execution
- Do not keep TODO-heavy placeholder code if a clean static section would be
better
- Do not stop with a plan only; execute the recovery

DELIVERABLES BEFORE STOPPING
You must finish with:
1. A short repo audit summary
2. The stack/framework/router detected
3. Exact files changed
4. Exact duplicate/dead files removed
5. Route-by-route status:
   - preserved
   - fixed
   - simplified
   - optional and kept
   - optional and reduced
6. Exact commands run
7. Exact build result
8. Exact lint/typecheck/test result if those scripts exist
9. Remaining known issues, only if they are real and verified

SUCCESS CRITERIA
The task is done only if:
- the app builds successfully,
- the core routes are clean and coherent,
- duplicates are removed,
- the UI is premium and readable,
```

```
- advanced concepts are labeled honestly,
- and you provide an exact final report.
```

Use this in **one recovery thread at a time**. OpenAI's Codex docs explicitly say to avoid having two threads modify the same files, and their best-practices guidance warns against running live threads on the same files without worktrees. [11]

Polish Codex prompt ready to copy

Only run this after the recovery build has already passed.

```
The recovery pass is complete and the build is green.
Now do a polish-only pass on the existing Pickaxe Capital / AI Habitat OS
website.

IMPORTANT
- Do not change frameworks.
- Do not add major new features.
- Do not expand scope.
- Do not introduce fake live functionality.
- Preserve the recovered route structure and component architecture.
- Keep all honesty labels: Live / Static / Prototype / Research / Future
Adapter.

POLISH GOAL
Make the site feel premium, intentional, and impressive while preserving
stability.

FOCUS AREAS
1. Typography
- Improve hierarchy, headings, labels, card titles, and body text readability
- Make the homepage and key pages easier to scan in under 30 seconds

2. Layout
- Refine spacing, panel rhythm, gutters, and alignment
- Make desktop feel like a command center
- Make mobile feel simplified but not broken

3. Shell quality
- Improve left nav / top bar / page header / context rail consistency
- Strengthen active route states and hover/focus states

4. Visual system
- Keep one accent strategy only
- Use clean cards, subtle depth, clean separators, and restrained motion
- Avoid gimmicky gradients, excessive blur, or low-contrast overlays
```

```
5. Route-specific polish
- Home: sharpen the brand hierarchy and route cards
- Vision Map: make the static architecture diagram elegant and clear
- Agents: improve card design and scanability
- Archive: improve search/filter affordances and row/card readability
- Staging: make status tables/checklists feel structured and useful
- Founder / CEO B pages: improve editorial readability and trust

6. Optional routes
- If optional routes exist, make them feel integrated into the same system
- If they are prototype pages, make that status visually clear without making
them feel abandoned

ACCESSIBILITY / RESTRAINT
- Preserve readable contrast
- Keep interactions keyboard-friendly
- Reduce non-essential motion
- No autoplay anything
- No fake terminal animations unless they are subtle and clearly decorative

FINAL VALIDATION
- Run the build again
- Run lint/typecheck if scripts exist
- Report exact files changed and exact command results
- Summarize polish changes by route and by shared component
```

QA checklist and acceptance standard

Use this checklist as the stopping gate for Codex and for your own review:

- [ ] The repository's actual build command completes successfully with zero build errors.
- [ ] If `lint` exists, it passes; if `typecheck` exists, it passes; if lightweight tests already exist, the relevant subset passes.
- [ ] The core routes all resolve cleanly: `/`, `/vision-map`, `/agents`, `/archive`, `/staging`, `/founder`, and `/ceo-b-profile`.
- [ ] Optional routes are either stable and integrated or intentionally simplified and honestly labeled.
- [ ] There is one shared site shell, not multiple competing shells.
- [ ] Navigation labels, route metadata, and status labels come from a single canonical source.
- [ ] Duplicate pages, duplicate components, or dead experimental copies have been removed.
- [ ] Advanced capabilities are never implied as live unless they are implemented and testable.
- [ ] The home page explains the brand hierarchy quickly and clearly.
- [ ] `/vision-map` explains the system architecture visually without pretending to be real telemetry.
- [ ] `/agents` shows roles and scopes without pretending to execute real agents.
- [ ] `/archive` is useful even with local/static data only.
- [ ] `/staging` acts as the single source of truth for build status, QA, and next steps.

- [ ] Text contrast is readable, with at least WCAG AA minimum contrast for normal text. [12]
- [ ] Pages expose sensible landmarks such as navigation and main content; if a right rail exists, it should behave like a true complementary/aside region. [13]
- [ ] Non-essential motion is reduced or disabled when users prefer reduced motion. [14]
- [ ] The layout avoids obvious unstable shifts during load; if web-vitals tooling already exists, a "good" target remains LCP $\leq$ 2.5s, INP $\leq$ 200 ms, and CLS $\leq$ 0.1. [15]
- [ ] Codex produces an exact closing report: framework detected, files changed, files removed, commands run, and final results.

"Done" should mean something stricter than "looks better." The project is done when it has **one clear identity, one stable shell, one route system, one token system, honest labels for anything experimental, and a verified passing build**. If a flashy feature had to be reduced to a stable static or prototype panel to achieve that, that is a success, not a compromise.

What to avoid doing next

After recovery, the biggest risk is falling back into the same prompt pattern that created the mess: giant speculative instructions, multiple overlapping threads, too many tools, and no durable repo rules. OpenAI's best-practices docs are unusually clear here: once a prompting pattern works, move the durable guidance into `AGENTS.md`; define practical repo layout, run commands, constraints, and "done" criteria there; keep Codex from operating on the same files in concurrent threads; and do not add tools or automations until the workflow is already reliable manually. [16]

Concretely, avoid these next moves:

- Do **not** run multiple Codex recovery threads against the same branch or same files.
- Do **not** go back to giant one-off prompts once this stabilizes; turn the working rules into a concise repo-level `AGENTS.md`. [17]
- Do **not** wire in every MCP or external tool "just because it might be useful"; OpenAI recommends adding tools only when they unlock a real repeated workflow. [18]
- Do **not** automate repeated work until the manual recovery workflow is stable and trustworthy. [17]
- Do **not** let optional labs outrank the core product routes.
- Do **not** rebrand the system again; keep the hierarchy fixed: Pickaxe Capital → AI Habitat OS → CEO B.
- Do **not** convert research inspiration into product claims. OpenClaw, Addy's Jarvis, `jarvis-mlx`, OpenJarvis, Microsoft JARVIS, and JARVIS-1 should remain reference material unless you later implement real adapters and can verify them. [19]

If you hold that line, the site can stop being an accumulation of ambitious prompts and become what you actually want: a coherent, stable, premium command-center brand surface with room to grow.

---

[1] [10] [11] https://developers.openai.com/codex/prompting
https://developers.openai.com/codex/prompting

[2] [19] https://github.com/openclaw/openclaw/blob/main/docs/index.md
https://github.com/openclaw/openclaw/blob/main/docs/index.md

3 https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide
https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide

4 7 https://react.dev/learn/choosing-the-state-structure
https://react.dev/learn/choosing-the-state-structure

5 https://developers.openai.com/blog/run-long-horizon-tasks-with-codex
https://developers.openai.com/blog/run-long-horizon-tasks-with-codex

6 https://nextjs.org/docs/app/getting-started/layouts-and-pages
https://nextjs.org/docs/app/getting-started/layouts-and-pages

8 https://tailwindcss.com/docs/theme
https://tailwindcss.com/docs/theme

9 https://www.typescriptlang.org/tsconfig/strict.html
https://www.typescriptlang.org/tsconfig/strict.html

12 https://www.w3.org/TR/WCAG20-TECHS/G18.html
https://www.w3.org/TR/WCAG20-TECHS/G18.html

13 https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Reference/Roles/navigation_role
https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Reference/Roles/navigation_role

14 https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/%40media/prefers-reduced-motion
https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/%40media/prefers-reduced-motion

15 https://web.dev/articles/vitals
https://web.dev/articles/vitals

16 17 18 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices